DEEP NEURAL NETWORKS (AIML ZG565) · COMPANION READER

Deep Feed-Forward Neural Networks

Why depth matters, how backpropagation works, and how to build and train a DFNN

Session 5 (24 May 2026) · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. Colour code: **blue** intuition; **amber** everyday picture; **green** worked example; **red** watch out; **purple** key takeaway.

1 What this session covers

Sessions 3–4 built complete learning systems with a *single* neuron. Each model drew exactly one straight decision boundary. Session 5 asks the obvious next question: what if the real relationship is curved, or has many folds? The answer is to *stack* neurons into layers, introduce **non-linear activation functions** between them, and train the whole stack with **backpropagation**. The result is a **Deep Feed-Forward Neural Network (DFNN)**, also called a **Multi-Layer Perceptron (MLP)** — the foundation of modern deep learning.

2 Why linear models are not enough

A single neuron with any activation always computes a decision boundary that is linear in the input space (a straight line in 2D, a hyperplane in higher dimensions). The **XOR problem** is the canonical proof that this is insufficient.

☞ The intuition

Two inputs, four input combinations: $(0,0) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$, $(1,1) \to 0$. The two 1-outputs sit at opposite corners of the unit square. No straight line can separate them from the two 0-outputs. A single neuron literally cannot express the XOR function regardless of how long it trains.

This limitation applies to all real-world tasks where the relationship is non-linear: image recognition (pixel intensities → object classes), natural language (words → sentiment), medical diagnosis (symptoms → disease). Linear models will systematically fail on these.

✓ Key takeaway

Linear models can only draw straight boundaries. XOR (and most real problems) are non-linearly separable. The fix: add hidden layers with non-linear activations.

3 Activation functions: the source of non-linearity

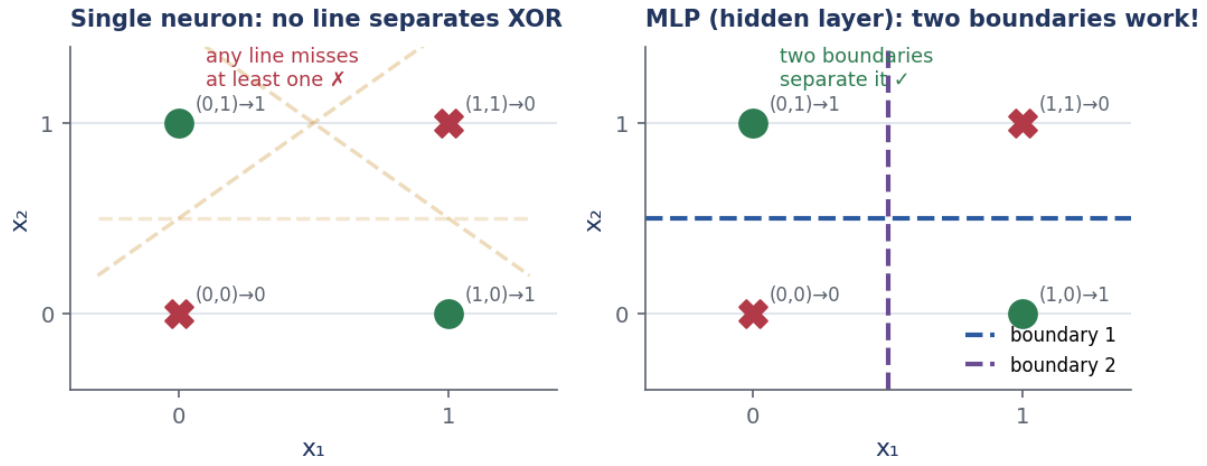

Figure 1: How to read it: left — any straight line misclassifies at least one XOR point. Right — two boundaries (provided by a hidden layer) separate the classes perfectly.

Without a non-linear activation function, stacking many linear layers is pointless: the composition of linear functions is still linear ($W_2(W_1x+b_1)+b_2 = (W_2W_1)x+\text{const}$). The activation function f applied element-wise between layers is what makes depth meaningful.

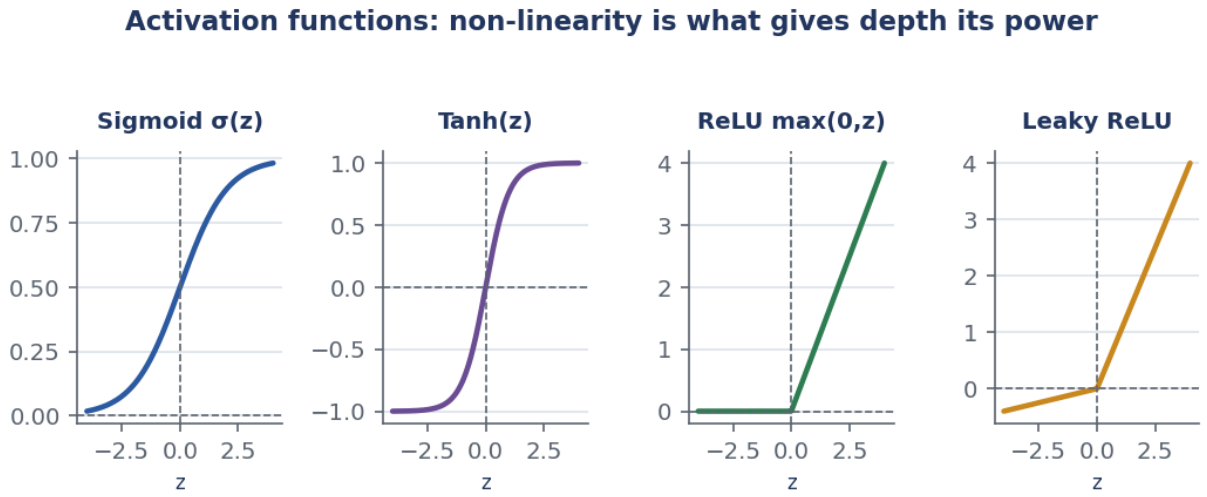

Figure 2: How to read it: four common activations, all non-linear. Sigmoid and tanh saturate at extremes (derivatives vanish). ReLU and Leaky ReLU maintain a gradient of 1 (or α) for positive inputs.

The four standard activations:

1. **Sigmoid** $\sigma(z) = 1/(1+e^{-z})$. Range $(0,1)$. Derivative $\sigma(z)(1-\sigma(z))$ peaks at 0.25, vanishes for large $|z|$. Use only at the output of a binary classifier.

2. **Tanh** $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$. Range $(-1,1)$. Zero-centred (unlike sigmoid), so gradients tend to not all have the same sign. Still saturates. Better than sigmoid in hidden layers, but largely replaced by ReLU.

3. **ReLU** $f(z) = \max(0,z)$. Gradient = 1 for $z > 0$, 0 for $z < 0$. No saturation on the positive side, so gradients flow freely through deep networks. Default hidden-layer activation in modern practice. Downside: "dying ReLU" — if a neuron's input is always negative, its gradient is always 0 and it never recovers.

4. **Leaky ReLU** $f(z) = \max(\alpha z, z)$ with $\alpha = 0.01$ typically. Solves the dying ReLU: the gradient is α (not 0) for $z < 0$.

★ Everyday picture

ReLU is like a one-way valve: if the signal is positive, pass it through unchanged; if negative, block it. The valve's derivative is always 1 (when open), so gradients in a deep network don't shrink as they flow backward — unlike sigmoid, which is more like a partially-closed valve that weakens the signal at every step.

✗ Watch out

The **vanishing gradient** problem: with sigmoid activations, the maximum gradient per layer is 0.25. In an L-layer network the gradient for the first layer is multiplied by 0.25^L — for $L = 8$ that is 1.5×10^{-5}, essentially zero. The first layers stop learning. ReLU avoids this because its gradient is 1 everywhere it is active.

✓ Key takeaway

Default for hidden layers: **ReLU**. For very deep nets consider Leaky ReLU. Never use sigmoid or tanh in hidden layers of a deep network. Reserve sigmoid for binary output; softmax for multi-class output.

4 DFNN architecture

A **Deep Feed-Forward Neural Network (DFNN)** has:

- An **input layer** — not counted as a computational layer.
- L computational layers: hidden layers $1, 2, \ldots, L-1$ plus the output layer L.
- Layer ℓ has n_ℓ neurons; weights $W^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$; bias $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$.

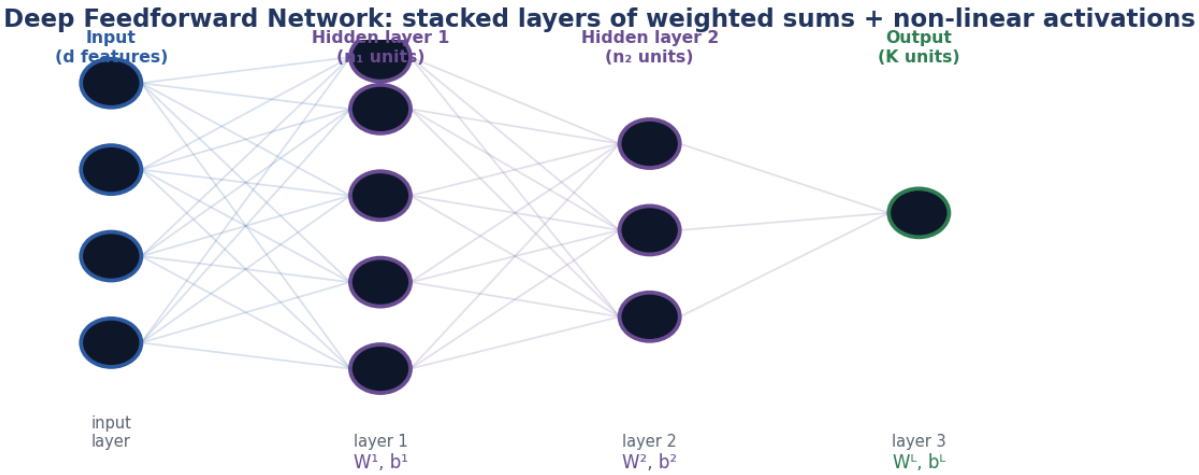

Figure 3: How to read it: arrows are weights. Each node in a layer receives input from all nodes in the previous layer. Information moves strictly forward (no loops). The output layer's activation depends on the task.

Layer-by-layer computation (forward pass): For each layer $\ell = 1, \ldots, L$:

$$\mathbf{z}^{(\ell)} = W^{(\ell)\top}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \tag{1}$$

$$\mathbf{h}^{(\ell)} = f^{(\ell)}\left(\mathbf{z}^{(\ell)}\right), \tag{2}$$

where $\mathbf{h}^{(0)} = \mathbf{x}$ (the input), $f^{(\ell)}$ is the activation function of layer ℓ, and $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$ is the prediction.

✎ Worked example: forward pass through a 2-hidden-layer network — student exam example

Architecture: input $d = 2$ (study hours, attendance), hidden layers of sizes 3 and 2, output size 1 (pass probability). Use ReLU in hidden layers, sigmoid at output.
Given (from slides): one student input $\mathbf{x} = [5, 90]^\top$ and (small illustrative) weights:

$$W^{(1)} = \begin{bmatrix} 0.1 & 0.2 & -0.1 \\ 0.05 & 0.1 & 0.15 \end{bmatrix}, \quad \mathbf{b}^{(1)} = [0.1, 0.0, -0.05]^\top.$$

Step 1. Layer 1 pre-activation.

$$z_1^{(1)} = 0.1 \times 5 + 0.05 \times 90 + 0.1 = 0.5 + 4.5 + 0.1 = \mathbf{5.1}. \tag{3}$$

$$z_2^{(1)} = 0.2 \times 5 + 0.1 \times 90 + 0.0 = 1.0 + 9.0 + 0.0 = \mathbf{10.0}. \tag{4}$$

$$z_3^{(1)} = -0.1 \times 5 + 0.15 \times 90 - 0.05 = -0.5 + 13.5 - 0.05 = \mathbf{12.95}. \tag{5}$$

Step 2. Layer 1 activation (ReLU). All pre-activations are positive, so ReLU passes them unchanged: $\mathbf{h}^{(1)} = [5.1, 10.0, 12.95]^\top$.

Step 3. Layer 2 (say output is $\hat{y} = \sigma(w_1 h_1^{(1)} + w_2 h_2^{(1)} + w_3 h_3^{(1)} + b)$ with $w = [0.1, -0.05, 0.08], b = -7$).

$$z^{(L)} = 0.1(5.1) + (-0.05)(10.0) + 0.08(12.95) - 7 \tag{6}$$

$$= 0.51 - 0.50 + 1.036 - 7 = \mathbf{-5.954}. \tag{7}$$

Step 4. Output sigmoid. $\hat{y} = \sigma(-5.954) = 1/(1 + e^{5.954}) \approx \mathbf{0.003}$. The model predicts a 0.3% chance of passing — with only 5 hours of study, this seems reasonable.

Step 5. Sense-check. Deeper study (e.g. 8 hours, 95% attendance) would produce larger hidden-layer values and a higher final sigmoid output.

Vectorised mini-batch computation. For a mini-batch of B examples, stack inputs row-wise into $X \in \mathbb{R}^{B \times d}$. Then:

$$Z^{(\ell)} = H^{(\ell-1)}W^{(\ell)} + \mathbf{b}^{(\ell)} \quad \in \mathbb{R}^{B \times n_\ell}, \tag{8}$$

$$H^{(\ell)} = f\left(Z^{(\ell)}\right) \quad \text{(applied element-wise)}. \tag{9}$$

One matrix multiply replaces B separate dot products. This is why GPUs can train networks with millions of parameters efficiently.

✓ Key takeaway

Forward pass: $\mathbf{z}^{(\ell)} = W^{(\ell)\top}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}$, then $\mathbf{h}^{(\ell)} = f(\mathbf{z}^{(\ell)})$, layer by layer. For B examples: one matrix multiply per layer.

5 Counting parameters

A network with d inputs, L hidden layers of sizes $n_1, \ldots, n_{L-1}$, and K outputs has:

$$\text{Parameters} = \sum_{\ell=1}^{L} \underbrace{n_{\ell-1} \times n_\ell}_{\text{weights}} + \underbrace{n_\ell}_{\text{biases}}, \tag{10}$$

where $n_0 = d$ and $n_L = K$.

✎ Worked example: parameter count for the student example

Architecture: $d = 2$ inputs, hidden sizes $n_1 = 3$, $n_2 = 2$, output $K = 1$.

Step 1. Layer 1. $2 \times 3 = 6$ weights $+3$ biases $=$ **9** parameters.

Step 2. Layer 2. $3 \times 2 = 6$ weights $+2$ biases $=$ **8** parameters.

Step 3. Output. $2 \times 1 = 2$ weights $+1$ bias $=$ **3** parameters.

Step 4. Total. $9 + 8 + 3 =$ **20** parameters.

Step 5. Sense-check. For a ResNet-50: 25,636,712 parameters; for GPT-4: ~ 1.8 trillion. The formula above scales all the way up.

✓ Key takeaway

Parameters $= \sum_\ell (n_{\ell-1} \times n_\ell + n_\ell)$. Each connection between adjacent layers contributes one weight; each neuron contributes one bias.

6 Backpropagation

Backpropagation is the algorithm that computes the gradient of the loss with respect to *every* parameter in the network efficiently. It is just the **chain rule** of calculus applied systematically, layer by layer, from output back to input.

☞ The intuition

Imagine the network is a long assembly line. A bad product comes out. You want to know which worker (parameter) was responsible. You trace the blame backwards: from the final inspector $\rightarrow$ the last station $\rightarrow$ the previous station $\rightarrow \cdots$. The chain rule is the mathematics of blame-tracing.

★ Everyday picture

A teacher prepares one answer key and uses it to check all 200 student papers at once. Backpropagation is the answer key: one backward pass computes all 200,000 gradients simultaneously, instead of computing each gradient separately (which would require 200,000 separate forward passes).

The algorithm. Define the **error signal** at layer ℓ as:

$$\delta^{(\ell)} = \frac{\partial J}{\partial Z^{(\ell)}}. \tag{11}$$

Step 1: Compute the error at the output layer from the loss gradient. Step 2: Propagate it backward using:

$$\delta^{(\ell)} = \left(\delta^{(\ell+1)} W^{(\ell+1)\top}\right) \odot f'^{(\ell)}\left(Z^{(\ell)}\right), \tag{12}$$

where $\odot$ is element-wise multiplication and f' is the derivative of the activation. Step 3: Compute parameter gradients from the error signals:

$$\frac{\partial J}{\partial W^{(\ell)}} = H^{(\ell-1)\top}\delta^{(\ell)}, \qquad \frac{\partial J}{\partial \mathbf{b}^{(\ell)}} = \sum_{\text{batch}} \delta^{(\ell)}. \tag{13}$$

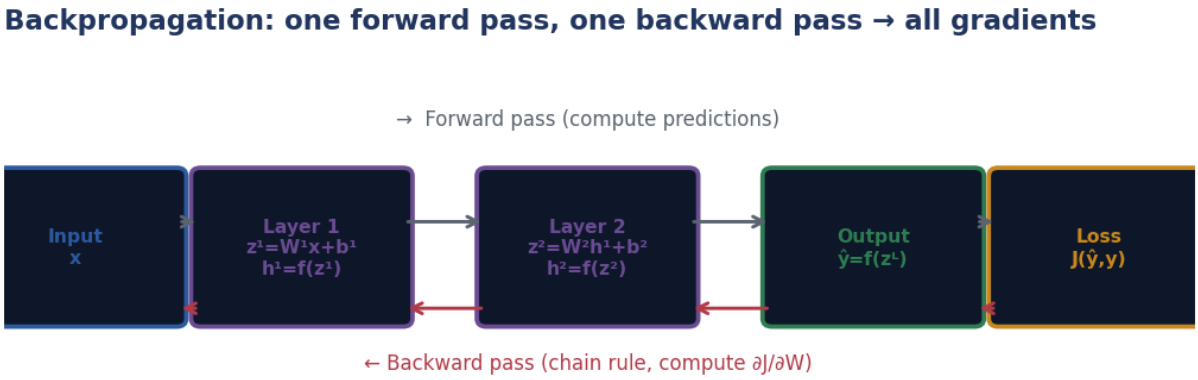

Figure 4: How to read it: the forward pass (top arrow) computes and stores intermediate values $H^{(\ell)}$ at each layer. The backward pass (bottom arrow) uses those stored values and the chain rule to compute gradients $\partial J/\partial W^{(\ell)}$ for every layer in one sweep.

Why backpropagation made deep learning possible. Computing all gradients naively requires one forward pass per parameter ($O(P^2)$ cost for P parameters). Backpropagation computes all P gradients in one forward + one backward pass ($O(P)$ cost). For a network with 200,000 parameters, this is a 200,000× speedup. Without it, training modern networks with millions of parameters would be computationally infeasible.

Dynamic programming connection. Backpropagation stores intermediate values ($Z^{(\ell)}, H^{(\ell)}$) during the forward pass and reuses them during the backward pass. This is exactly the "compute once, store, reuse" pattern of dynamic programming.

✗ Watch out

The **vanishing gradient** problem: if sigmoid activations are used in hidden layers, $f'(z) = \sigma(z)(1-\sigma(z)) \leq 0.25$. In an L-layer network, gradients shrink by up to 0.25^L at each backward step. For $L = 10$, that is $\sim 10^{-6}$. The early layers receive no useful gradient signal and stop learning. Solution: use ReLU ($f'(z) = 1$ for $z > 0$), which keeps gradients from shrinking.

✎ Worked example: one backpropagation step by hand on a tiny network

Single hidden layer with 1 hidden neuron (ReLU) and 1 output (sigmoid). Weights: $w_1 = 0.5$ (input-to-hidden), $w_2 = 0.8$ (hidden-to-output), biases both 0. Input $x = 2$, true label $y = 1$. Loss = binary cross-entropy.

Step 1. Forward pass. $z^{(1)} = 0.5 \times 2 = 1.0$; $h = \text{ReLU}(1.0) = 1.0$.
$z^{(2)} = 0.8 \times 1.0 = 0.8$; $\hat{y} = \sigma(0.8) \approx 0.6900$.

Step 2. Loss. $J = -\log(0.6900) \approx \mathbf{0.3711}$.

Step 3. Output error. $\delta^{(2)} = \hat{y} - y = 0.6900 - 1 = \mathbf{-0.3100}$.

Step 4. Gradient for w_2. $\partial J/\partial w_2 = h \cdot \delta^{(2)} = 1.0 \times (-0.3100) = \mathbf{-0.3100}$.

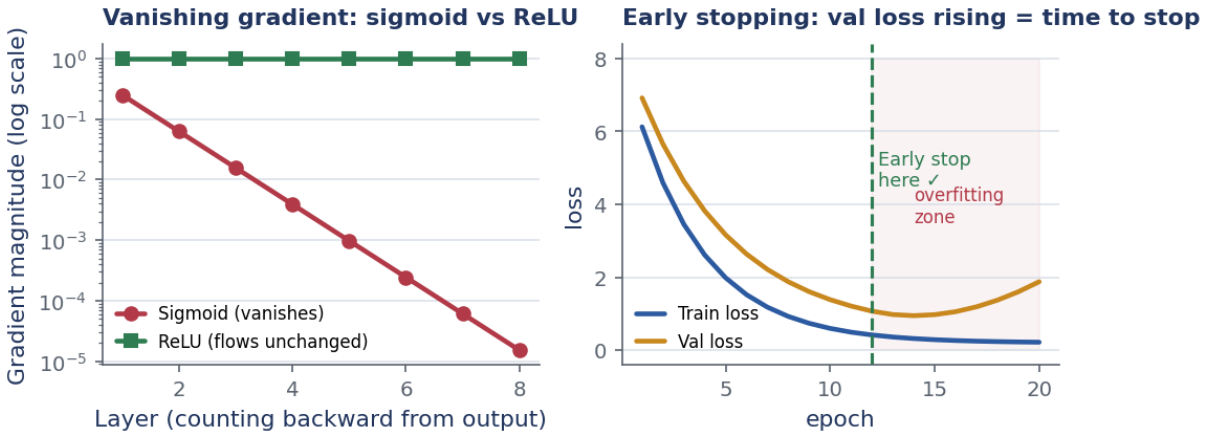

Figure 5: Left: gradient magnitude per layer under sigmoid (red, exponentially vanishing) vs ReLU (green, constant 1). Right: early stopping — the validation loss starts rising at epoch 12 while the training loss continues to fall; stop at epoch 12.

Step 5. Backpropagate error. $\delta^{(1)} = \delta^{(2)} \times w_2 \times \text{ReLU}'(z^{(1)}) = (-0.3100)(0.8)(1) = \mathbf{-0.2480}$. (ReLU derivative is 1 since $z^{(1)} = 1.0 > 0$.)

Step 6. Gradient for w_1. $\partial J/\partial w_1 = x \cdot \delta^{(1)} = 2.0 \times (-0.2480) = \mathbf{-0.4960}$.

Step 7. Update with $\eta = 0.1$. $w_2 \leftarrow 0.8 - 0.1(-0.3100) = \mathbf{0.8310}$; $w_1 \leftarrow 0.5 - 0.1(-0.4960) = \mathbf{0.5496}$.

Step 8. New prediction. $z^{(1)} = 0.5496 \times 2 = 1.099$; $h = 1.099$; $z^{(2)} = 0.8310 \times 1.099 = 0.913$; $\hat{y} = \sigma(0.913) \approx \mathbf{0.714}$. Better: moved from 0.69 toward 1.

✓ Key takeaway

Backpropagation = chain rule applied backwards. One forward pass (stores $H^{(\ell)}, Z^{(\ell)}$) + one backward pass = gradients for all P parameters. Complexity $O(P)$, not $O(P^2)$.

7 Depth vs width, and the Universal Approximation Theorem

Depth vs width. There are two ways to increase a network's capacity:

- **Deeper** (more layers): learns **hierarchical features** — edges → textures → parts → objects. More expressive per parameter. ResNets, transformers are deep.
- **Wider** (more units per layer): learns more **parallel features** at one abstraction level. Higher capacity but parameter count grows quadratically with width. Higher overfitting risk.

Universal Approximation Theorem (Cybenko 1989; Hornik 1989): a feed-forward network with *one* hidden layer and enough neurons can approximate any continuous function on a compact set, given an appropriate (non-linear) activation. In theory, even shallow networks are fully general. In practice, deep networks achieve the same approximation with *exponentially fewer* parameters and generalise better because they learn reusable hierarchical representations.

✓ Key takeaway

UAT: one hidden layer can approximate any function (in theory). In practice: deeper is more parameter-efficient and generalises better. Start with 2–3 hidden layers and grow from there.

8 Challenges with deep networks and solutions

- **Vanishing gradients**: sigmoid/tanh activations shrink gradients exponentially. **Solution**: ReLU (or Leaky ReLU), proper initialization, batch normalization.
- **Exploding gradients**: gradients grow exponentially in early layers, causing NaN/Inf losses. **Solution**: **gradient clipping** (cap gradient norm at a threshold), proper initialization.
- **Degradation problem**: very deep networks become harder to optimise; training error itself increases. **Solution**: **residual connections** (skip connections, ResNets) that let gradients flow around layers.
- **Overfitting**: the model memorises training data and fails on test data. **Solutions**: L2 regularisation, dropout, early stopping, more data.

Regularisation:

- **L2 (weight decay):** add $\frac{\lambda}{2}\|W\|^2$ to the loss. Penalises large weights, forcing the model toward simpler solutions.
- **Dropout:** randomly set a fraction of neuron outputs to 0 during training. Forces the network to not rely on any single neuron — every neuron must be redundantly useful.
- **Early stopping:** monitor validation loss; stop training as soon as it starts rising.

Initialisation:

- **Zero init is wrong:** all neurons compute the same gradient, learning the same feature. Use random init.
- **Xavier/Glorot init:** designed for sigmoid/tanh, scales variance as $1/n_{\text{in}}$.
- **He init:** designed for ReLU, scales variance as $2/n_{\text{in}}$.
- **Biases**: initialise to zero.

✓ Key takeaway

Deep networks need: ReLU activations, He init, mini-batch SGD, dropout, and early stopping. In that combination, networks with millions of parameters train reliably.

9 Design guidelines and diagnostics

Output layer design: regression → identity activation + MSE; binary → sigmoid + BCE; multi-class → softmax + categorical CE.

Architecture guidelines: start with 2–3 hidden layers and 10–100 units each. Scale with data: more data → can support a larger network. Prefer decreasing widths (funnel architecture) for compression tasks.

Diagnosing training problems:

Symptom	Diagnosis	Fix
Loss = NaN/Inf	Numerical instability	Decrease η; normalise inputs
Loss not falling	η too small, or bug	Increase η; check grad computation
Loss oscillating	η too large	Decrease η
Train $\downarrow$, val $\uparrow$	Overfitting	Add regularisation; more data
Both losses high	Underfitting	Wider/deeper; train longer
Gradients $\to 0$	Vanishing gradients	ReLU; better init; batch norm
Gradients $\to \infty$	Exploding gradients	Gradient clipping; decrease η

✓ Key takeaway

The training loop: **(1)** check shapes, **(2)** overfit a single mini-batch (loss should reach 0), **(3)** add regularisation, **(4)** monitor train and val loss separately.

10 Recap and what comes next

One paragraph. Linear models fail on non-linear problems (XOR). The fix is a DFNN: stacked layers of (weighted sum + non-linear activation). ReLU is the standard hidden-layer activation because it avoids vanishing gradients. The forward pass computes $\mathbf{h}^{(\ell)} = f(W^{(\ell)\top}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)})$ for each layer. Back-propagation efficiently computes all gradients in one backward pass using the chain rule. Parameters $= \sum_{\ell}(n_{\ell-1}n_{\ell} + n_{\ell})$. Depth enables hierarchical feature learning; width adds parallel capacity. Training requires proper init, normalisation, and regularisation.

✓ Key takeaway

DFNN = linear models + hidden layers + non-linear activations + backpropagation. This is the foundation all subsequent sessions build on: CNNs, RNNs, attention, and transformers are all specialisations of this framework.

Further reading. Zhang et al. *Dive into Deep Learning*, Chapters 4–5; Goodfellow et al., Chapter 6; Bishop, Chapter 5.